SLIDE 1

Métricas e parâmetros de desempenho

Visão geral da aula

Nesta aula estudaremos como medir desempenho em sistemas computacionais. Vamos diferenciar métrica, parâmetro e carga de trabalho, depois aplicar tempo de resposta, latência, vazão, utilização, disponibilidade, eficiência e escalabilidade em exemplos de redes, bancos de dados e aplicações web.



SLIDE 2

Por que medir desempenho?

Avaliação antes da decisão técnica

Medir desempenho permite comparar alternativas com base em dados, não apenas em impressão. Um professor, por exemplo, pode perguntar se um servidor novo realmente melhora o Moodle. Para responder, precisamos observar indicadores objetivos antes, durante e depois da mudança.

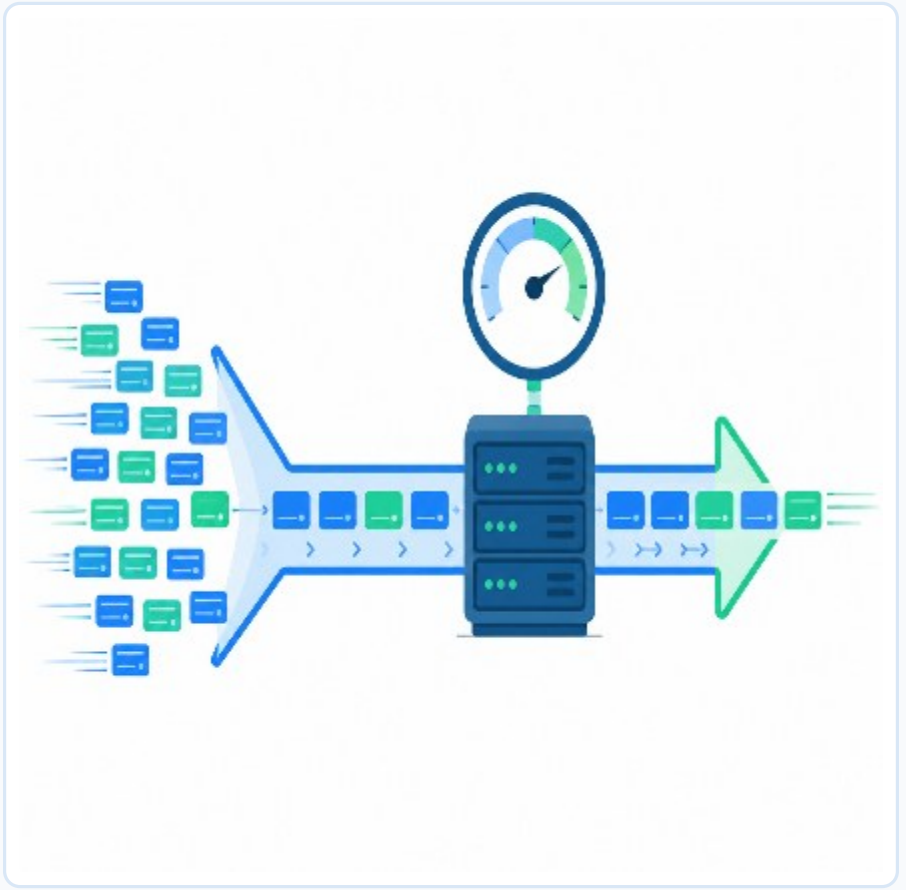


SLIDE 3

Sistema computacional como serviço

Pedidos, processamento e respostas

Um sistema computacional recebe requisições, executa algum processamento e devolve respostas. A requisição pode ser uma consulta SQL, um pacote de rede, um clique em uma página ou uma tarefa de impressão. O desempenho depende desse fluxo completo.



SLIDE 4

Métrica: o que será medido

Critério quantitativo de comparação

Métrica é o indicador usado para quantificar o desempenho. Em uma aplicação web, pode ser o tempo até abrir uma página; em rede, a vazão em Mbps; em banco de dados, consultas por segundo. A métrica precisa combinar com o objetivo do estudo.



SLIDE 5

Parâmetro: o que será variado

Fatores que influenciam a métrica

Parâmetro é aquilo que o avaliador modifica ou controla durante o teste. Podemos variar número de usuários, tamanho da requisição, hardware do servidor, versão do banco de dados ou largura de banda. Bons parâmetros têm relação direta com a métrica observada.



SLIDE 6

Carga de trabalho

O volume e o tipo de uso do sistema

Carga de trabalho é o conjunto de requisições que o sistema precisa atender. Ela deve representar o uso real. Testar um sistema de matrícula com 20 acessos leves não ajuda se, no período real, centenas de alunos acessam simultaneamente.



SLIDE 7

Tempo de resposta

Do pedido até a resposta chegar

Tempo de resposta é o intervalo entre o usuário solicitar um serviço e receber o resultado. Em uma consulta ao banco, começa no envio da consulta e termina na chegada da resposta. Quanto menor o tempo de resposta, melhor é a experiência percebida.

Tempo de resposta = instante da resposta – instante da solicitação



SLIDE 8

Latência

Atraso antes ou durante a comunicação

Latência é o atraso percebido na transmissão ou processamento. Em redes, inclui propagação, roteadores e filas. Em aplicações web, pode aparecer antes do servidor começar a responder. Uma página pode ter baixa vazão e ainda parecer lenta por latência alta.

Latência alta = maior espera antes de dados úteis chegarem

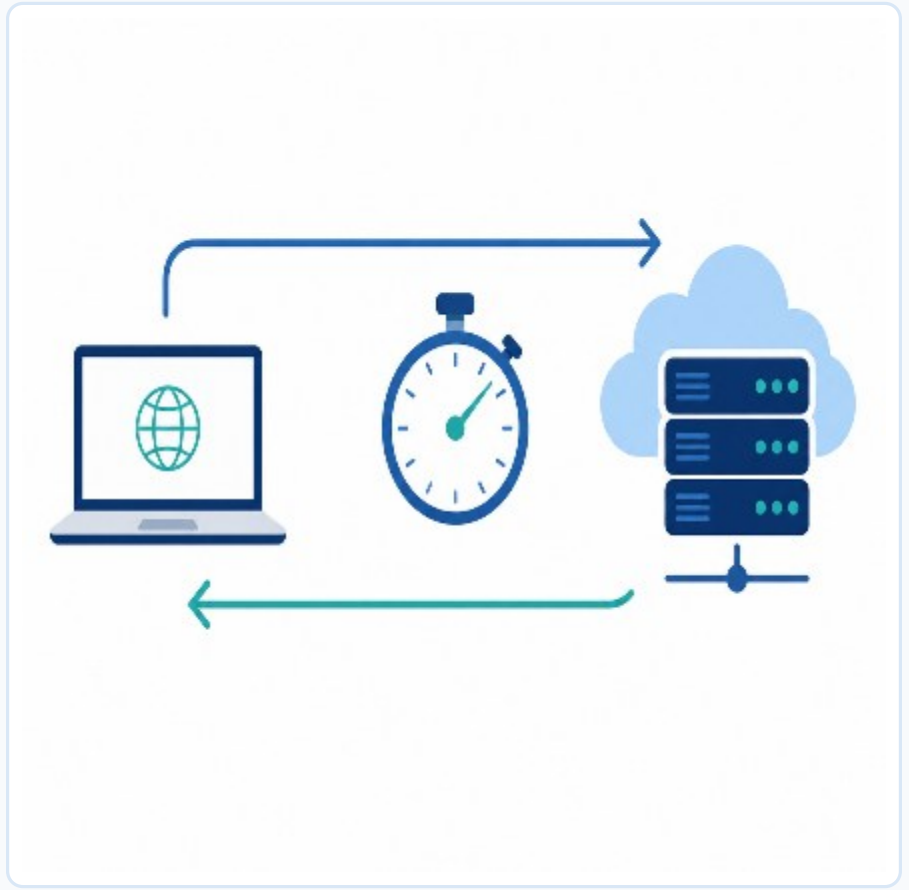


SLIDE 9

Latência média e percentis

Média pode esconder usuários lentos

A média resume o comportamento geral, mas pode esconder casos ruins. Se 95% das requisições respondem até 800 ms, dizemos que o p95 é 800 ms. Percentis ajudam a entender a experiência dos usuários mais afetados pela lentidão.



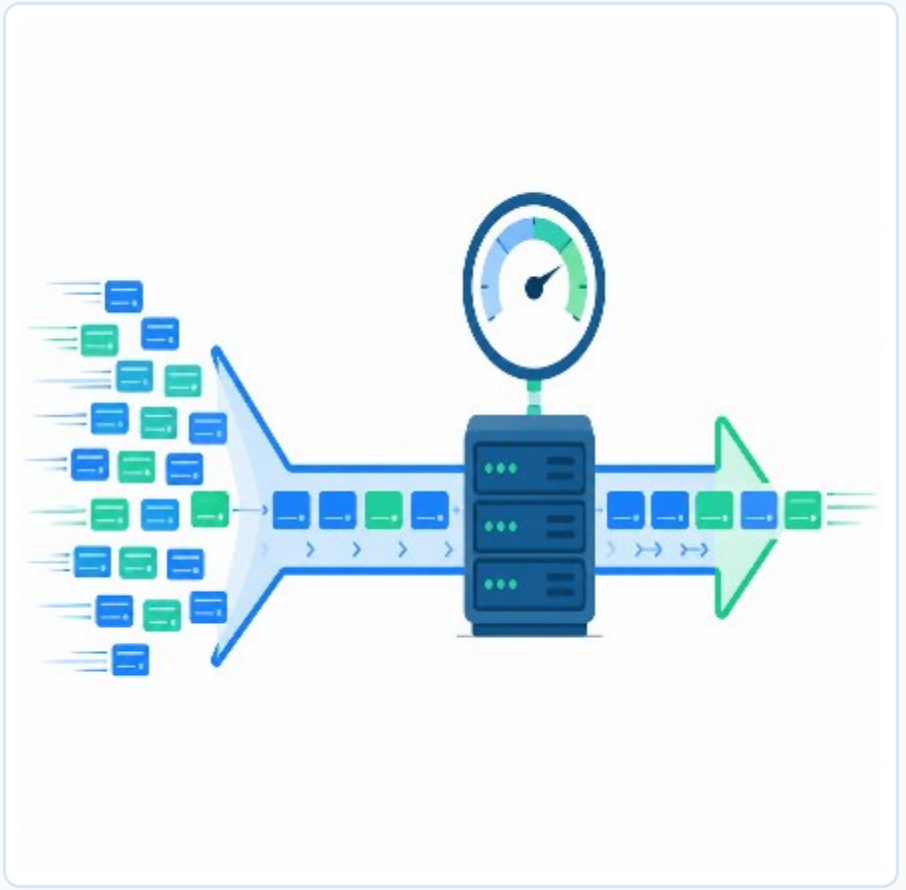
SLIDE 10

Vazão ou throughput

Quantidade atendida por unidade de tempo

Vazão mede quantas requisições, pacotes, transações ou operações são atendidas em determinado intervalo. Em rede usamos bps ou Mbps; em banco de dados, consultas por segundo; em servidor web, requisições por segundo.

Vazão = quantidade de pedidos atendidos / tempo observado



SLIDE 11

Exemplo de cálculo de vazão

Comparando duas máquinas

Imagine dois servidores processando a mesma API. O servidor A conclui 300 requisições em 60 segundos; o servidor B conclui 420 no mesmo período. A vazão de A é 5 req/s e a de B é 7 req/s, logo B processa mais.

A = 300/60 = 5 req/s | B = 420/60 = 7 req/s



SLIDE 12

Utilização de recursos

Fração do tempo ocupado

Utilização mostra quanto um recurso permanece ocupado. CPU a 80% significa que ela ficou ocupada em grande parte do período observado. Utilização muito baixa pode indicar desperdício; muito alta pode indicar risco de fila e degradação.

Utilização = tempo ocupado / tempo total × 100



SLIDE 13

Exemplo de utilização

Servidor observado por 24 horas

Um servidor ficou ligado durante 24 horas, mas atendeu requisições de forma efetiva por 18 horas. A utilização observada foi 18 dividido por 24, isto é, 0,75 ou 75%. O número ajuda a avaliar capacidade e ociosidade.

U = 18 / 24 = 0,75 = 75%



SLIDE 14

Disponibilidade

Sistema funcionando quando necessário

Disponibilidade indica a proporção de tempo em que o serviço está acessível e funcionando. Um sistema pode ser rápido, mas pouco útil se fica fora do ar. Em aplicações críticas, pequenas interrupções geram impacto grande nos usuários.

Disponibilidade = tempo disponível / tempo total × 100



SLIDE 15

Eficiência

Resultado obtido com menos recurso

Eficiência relaciona desempenho e consumo de recursos. Um sistema eficiente atende bem usando menos CPU, memória, energia ou custo de nuvem. Não basta ser rápido: é importante entregar resultado sem desperdiçar infraestrutura.

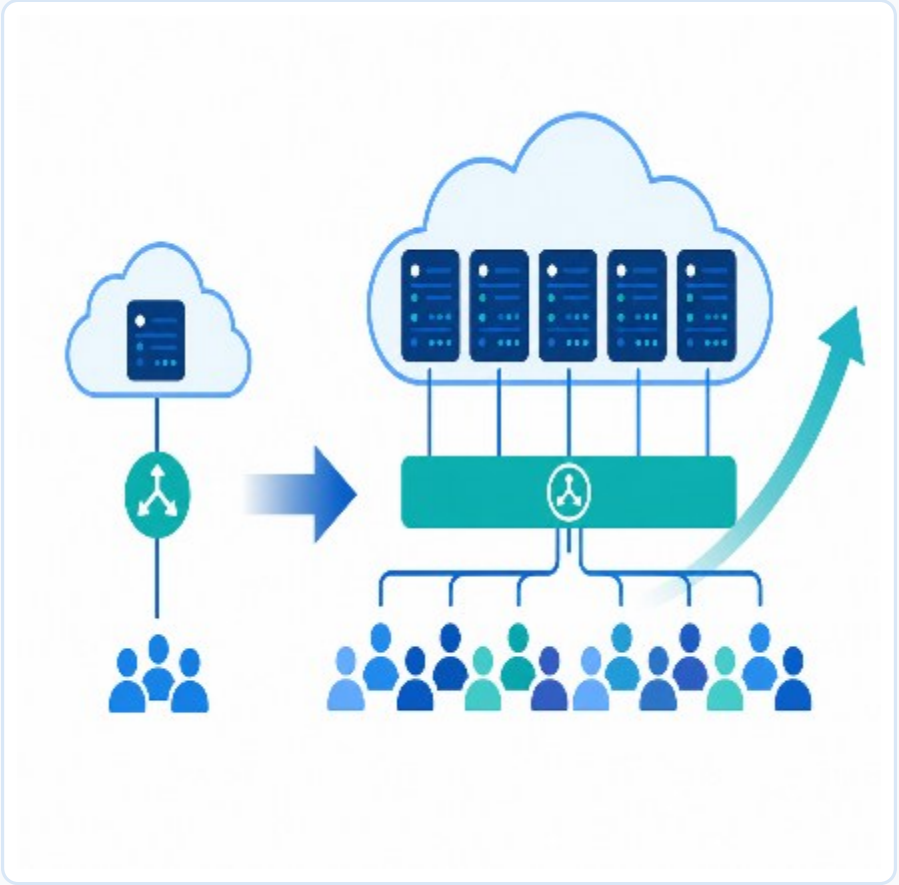


SLIDE 16

Escalabilidade

Crescer sem perder qualidade

Escalabilidade é a capacidade de manter desempenho aceitável quando a carga aumenta. Um sistema escalável consegue atender mais usuários com ajuste de arquitetura, como balanceamento, cache, replicação de banco ou aumento de servidores.

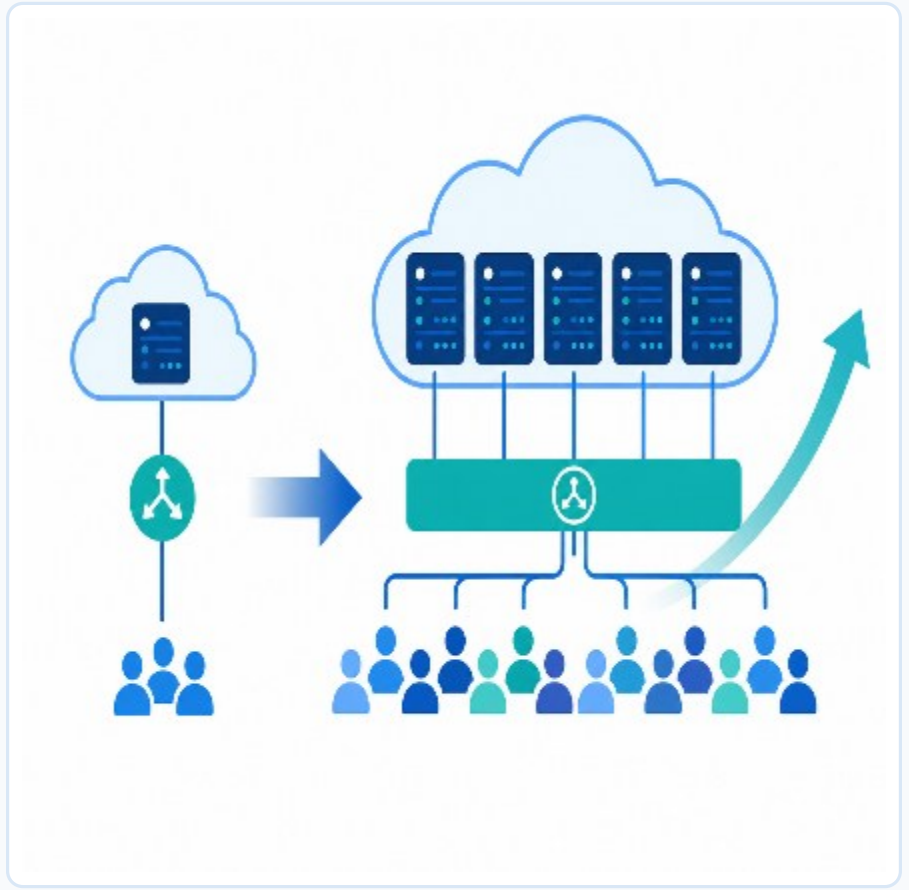


SLIDE 17

Escala vertical e horizontal

Duas formas de aumentar capacidade

Na escala vertical, aumentamos recursos da mesma máquina: mais CPU, memória ou disco. Na escala horizontal, adicionamos novas máquinas e distribuímos requisições. A escolha depende de custo, arquitetura, banco de dados e tipo de carga.



SLIDE 18

Relação entre métricas

Uma métrica afeta a outra

As métricas não devem ser analisadas isoladamente. Quando a vazão aumenta, a utilização também pode subir. Se o recurso se aproxima do limite, filas aparecem e o tempo de resposta cresce. Por isso, desempenho exige leitura conjunta.



SLIDE 19

Gargalo de desempenho

O recurso que limita o sistema

Gargalo é o componente que limita o fluxo geral. Pode ser CPU, memória, disco, rede, banco de dados ou uma chamada externa. Identificar gargalos evita trocar peças erradas e permite atacar a causa principal da lentidão.

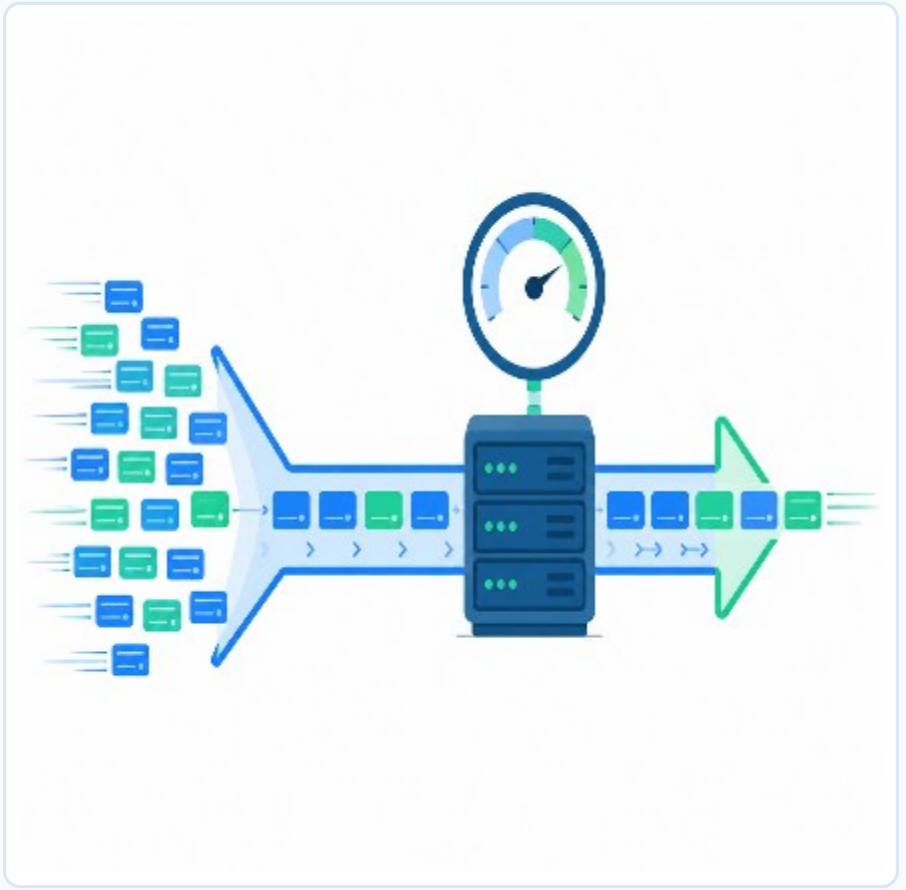


SLIDE 20

Filas e espera

Quando a chegada supera o atendimento

Filas surgem quando requisições chegam mais rápido do que o sistema consegue atender. Mesmo com servidor funcionando, a experiência piora porque pedidos aguardam processamento. Em sistemas web, filas podem ocorrer no banco, no servidor ou na rede.



SLIDE 21

Lei de Little em linguagem simples

Clientes no sistema, chegada e resposta

A Lei de Little relaciona número médio de itens no sistema, taxa de chegada e tempo médio de resposta. Se chegam muitas requisições e cada uma demora mais, haverá mais requisições dentro do sistema ao mesmo tempo.



L = λ × W | itens no sistema = taxa de chegada × tempo médio

SLIDE 22

SLO, SLA e indicadores

Metas de desempenho e compromisso

SLO é uma meta interna de nível de serviço, como “95% das requisições abaixo de 1 segundo”. SLA é um compromisso formal com cliente. Indicadores como disponibilidade, latência e erro ajudam a verificar se essas metas foram cumpridas.



SLIDE 23

Monitoramento contínuo

Coletar dados durante a operação

Monitoramento registra métricas ao longo do tempo. Ferramentas como Prometheus e Grafana ajudam a observar uso de CPU, memória, latência, erros, vazão e disponibilidade. O objetivo é detectar tendência, anomalia e risco antes da falha.



Teste de carga

Simular usuários para observar limite

Teste de carga simula usuários ou requisições para observar como o sistema se comporta. Ele ajuda a descobrir se a aplicação suporta a demanda esperada, qual recurso satura primeiro e em que ponto o tempo de resposta deixa de ser aceitável.



SLIDE 25

Passo a passo com k6

Exemplo prático gratuito

Um teste simples pode ser feito com k6. Primeiro instale a ferramenta. Depois crie um arquivo JavaScript com a URL testada. Em seguida defina usuários virtuais e duração. Por fim execute o teste e observe tempo de resposta, erros e requisições por segundo.



SLIDE 26

Passo a passo com JMeter

Alternativa visual para laboratório

No JMeter, crie um Test Plan, adicione Thread Group, configure número de usuários, período de subida e duração. Depois adicione HTTP Request e Summary Report. Execute o teste e analise média, mínimo, máximo, erros e throughput.



SLIDE 27

DevTools do navegador

Medição rápida no cliente

O navegador também permite observar desempenho. No Chrome ou Edge, abra F12, acesse Network, recarregue a página e veja tempo de carregamento, tamanho dos arquivos e status das requisições. É uma forma simples de iniciar a investigação.



SLIDE 28

Planejamento do experimento

Antes de medir, defina o método

Um bom experimento define objetivo, métrica, parâmetros, carga, ambiente, repetição e critério de comparação. Sem planejamento, os números podem confundir. Medir apenas uma vez, em ambiente instável, pode levar a conclusões incorretas.

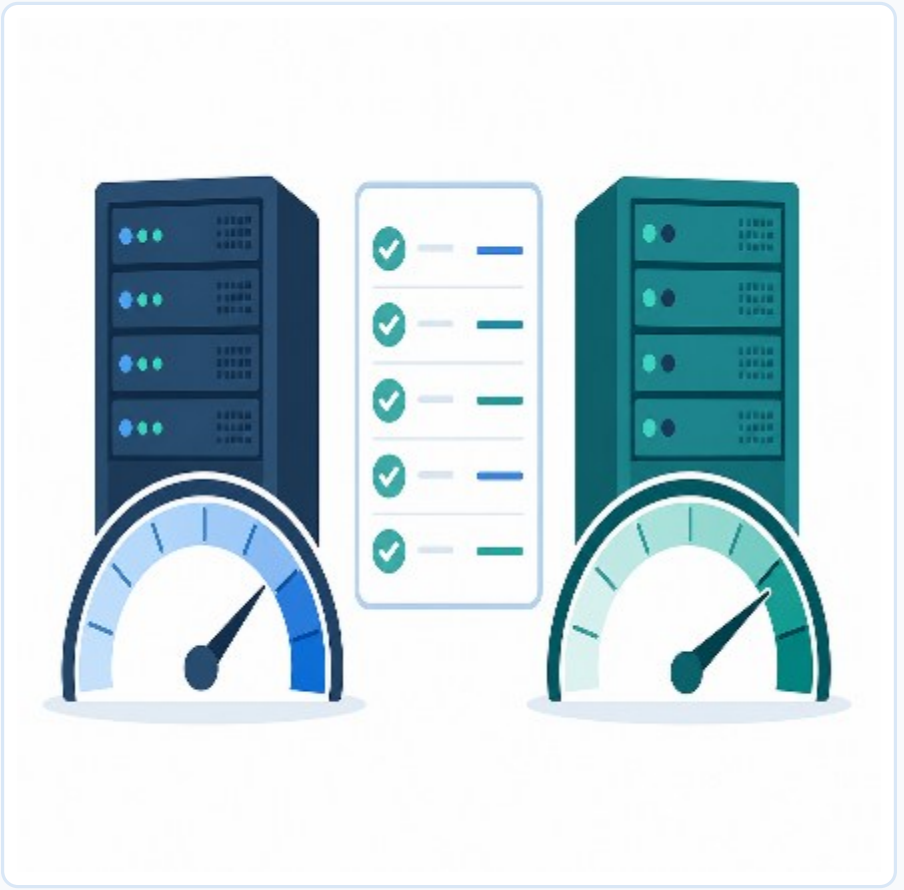


SLIDE 29

Coleta e organização dos dados

Registrar para comparar

Os resultados devem ser organizados em tabela. Registre cenário, horário, número de usuários, média, p95, throughput, erros e utilização. Essa organização permite comparar versões, identificar tendência e justificar tecnicamente uma recomendação.

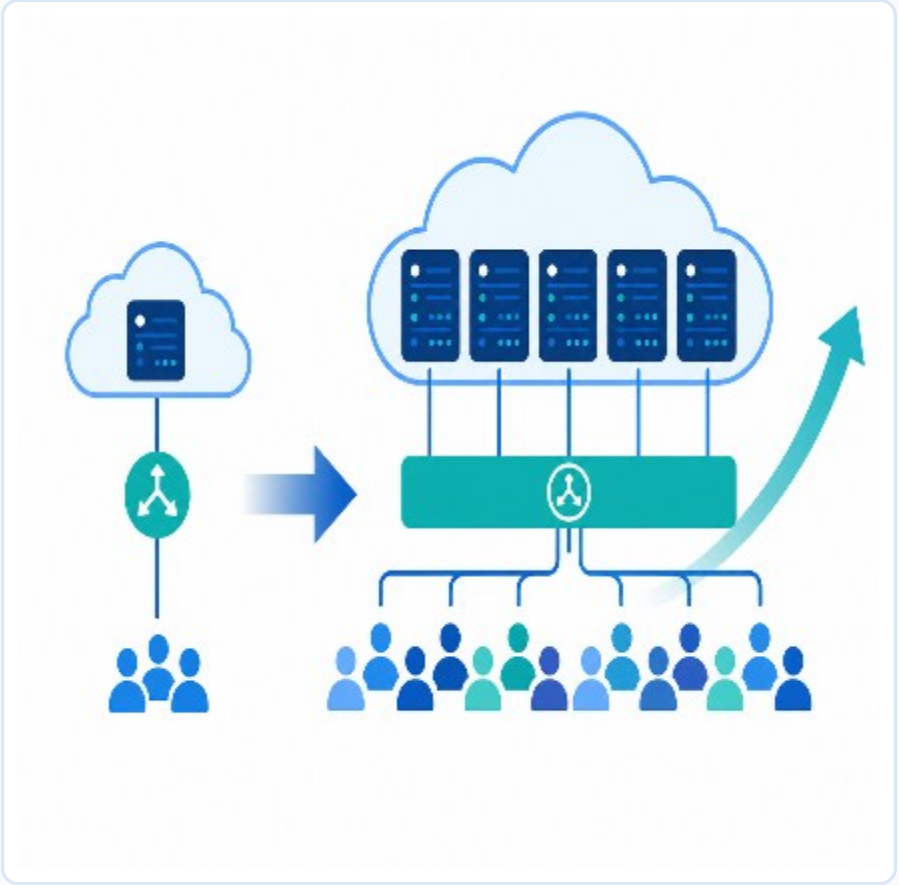


SLIDE 30

Exemplo integrado: portal acadêmico

Aplicando várias métricas

Imagine um portal acadêmico em semana de matrícula. A equipe mede 1.000 acessos simultâneos, tempo médio de 1,8 s, p95 de 5 s, CPU a 92% e erros ocasionais. A conclusão provável é que há risco de saturação e necessidade de ajuste.



SLIDE 31

Comparação justa entre sistemas

Mesma carga, mesmo método

Comparar sistemas exige condições equivalentes. Se o servidor A foi testado com 100 usuários e o B com 400, a comparação não é justa. Use a mesma carga, mesma duração, mesma ferramenta e ambiente semelhante para reduzir viés.



SLIDE 32

Erros comuns de interpretação

Números sem contexto enganam

Um throughput alto pode vir acompanhado de muitos erros. Uma baixa utilização pode ocorrer porque o sistema nem recebeu carga suficiente. Uma média boa pode esconder picos ruins. Por isso, sempre leia métricas em conjunto e com contexto.

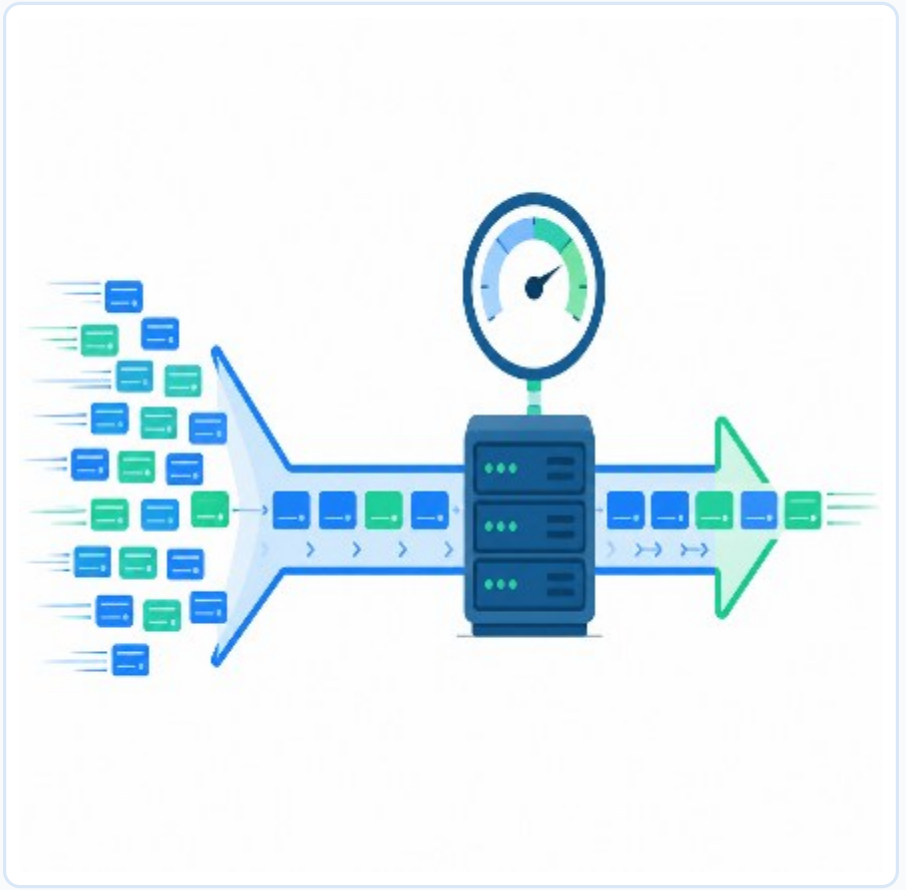


SLIDE 33

Atividade objetiva 1

Questões para fixação

1) Qual métrica mede pedidos atendidos por segundo? 2) Qual indicador representa a fração do tempo em que o recurso fica ocupado? 3) Em uma API, por que p95 pode ser mais informativo que média simples?



SLIDE 34

Atividade objetiva 2

Cálculo rápido em sala

Um servidor respondeu 900 requisições em 3 minutos. Durante o teste, a CPU ficou ocupada 135 segundos em um total de 180 segundos. Calcule a vazão em requisições por segundo e a utilização percentual da CPU.



Vazão = 900/180 = 5 req/s | U = 135/180 = 75%

SLIDE 35

Estudo de caso orientado

Recomendar melhoria com base em métricas

Um sistema de biblioteca fica lento quando muitos alunos buscam livros. A equipe registrou p95 de 7 s, CPU a 45%, banco com consultas lentas e vazão instável. A recomendação inicial deve focar no banco, não na troca do servidor web.



SLIDE 36

Síntese da aula

Como avaliar desempenho com critério

Avaliar desempenho significa definir objetivo, escolher métricas, controlar parâmetros, aplicar carga representativa e interpretar resultados com cuidado. Tempo de resposta, latência, vazão, utilização, disponibilidade, eficiência e escalabilidade formam um conjunto de indicadores para decisões técnicas melhores.



SLIDE 37

Referências e materiais

Fontes usadas e sugestões de consulta

JOHNSON, Thienne; COUTINHO, Mauro. Avaliação de Desempenho de Sistemas Computacionais. LTC, 2011. Apache JMeter User Manual. Grafana k6 Documentation. Prometheus Documentation. Google Cloud Observability: SLO monitoring. Sugestão: pesquisar vídeos introdutórios sobre JMeter, k6, Prometheus e Grafana.

